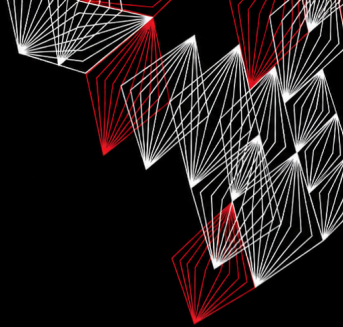


Tema 102: Instalación de Linux y gestión de paquetes

Erick Ricardo Martinez Martinez
Luis Fernando Torres Hernández
Laboratorio de Supercómputo y
Visualización en Paralelo
Junio 03, 2026



Contenido

- 1 102.1 Diseño del esquema de particionado del disco duro
- 2 102.2 Instalar un gestor de arranque
- 3 102.3 Gestión de librerías compartidas
- 4 102.4 Gestión de paquetes Debian
- 5 102.5 Gestión de paquetes RPM y YUM
- 6 102.6 Linux como sistema virtualizado

102.1 Diseño del esquema de particionado del disco duro

Conceptos Clave

- ▶ Un disco es un "contenedor físico". Antes de usarlo, necesita ser **particionado**.
- ▶ Una **partición** es un subconjunto lógico del disco.
- ▶ Dentro de cada partición hay un **sistema de archivos**.
- ▶ **LVM** (Logical Volume Manager) permite una gestión más flexible del almacenamiento.

Puntos de Montaje

Montaje

Para acceder a un sistema de archivos en Linux, debe ser **montado** en un directorio llamado **punto de montaje**.

- ▶ `/media`: Punto de montaje predeterminado para medios extraíbles (USB, CD-ROM, etc.).
- ▶ `/mnt`: Directorio tradicional para montajes temporales manuales.

Ejemplo

Una memoria USB con la etiqueta `FlashDrive` conectada por el usuario `john` se monta en `/media/john/FlashDrive/`.

Manteniendo las cosas separadas: Particiones Esenciales

Razones para particiones separadas

- ▶ **Seguridad y Recuperación:** Aislar el gestor de arranque (/boot) de fallos en la partición raíz.
- ▶ **Reinstalación:** Mantener los datos de usuario en /home facilita reinstalar el sistema.
- ▶ **Estabilidad:** Evitar que procesos llenen la partición raíz (/) separando /var.
- ▶ **Rendimiento:** Usar discos rápidos para el sistema y lentos para datos masivos.

Particiones Esenciales: /boot y /home

/boot

- ▶ Contiene archivos del gestor de arranque (GRUB), el kernel y el disco RAM inicial (initramfs).
- ▶ Asegura el arranque incluso si la partición raíz falla o usa un sistema de archivos no soportado por el bootloader.
- ▶ Su ubicación al inicio del disco asegura compatibilidad con sistemas BIOS antiguos (límite del cilindro 1024).
- ▶ Tamaño recomendado: **300 MB**.

/home

- ▶ Almacena los directorios de inicio de los usuarios.
- ▶ Separarlo facilita las reinstalaciones y actualizaciones del sistema sin perder datos personales.

Particiones Esenciales: /var y Swap

/var

- ▶ Contiene "datos variables": logs (/var/log), archivos temporales (/var/tmp) y cachés.
- ▶ Separarlo protege la partición raíz de que se llene por procesos anómalos.
- ▶ Es crucial para servidores (Apache, MySQL) donde puede crecer mucho.

Espacio de Intercambio (Swap)

- ▶ Utiliza espacio en disco como memoria virtual para la RAM.
- ▶ Se configura con mkswap y swapon.
- ▶ El tamaño recomendado varía según la cantidad de RAM y el uso (hibernación).

Introducción a LVM (Logical Volume Manager)

LVM

Es una forma de **virtualización de almacenamiento** que ofrece una gestión más flexible que el particionado tradicional.

Conceptos Clave

- ▶ **Physical Volume (PV)**: Un dispositivo de bloque (disco o partición).
- ▶ **Volume Group (VG)**: Agrupa uno o más PVs, creando un único "pool" de almacenamiento.
- ▶ **Logical Volume (LV)**: Una "partición lógica" dentro de un VG.

Ventaja

Permite redimensionar volúmenes lógicos fácilmente, agregando o quitando espacio de un VG sin necesidad de mover datos o redimensionar particiones físicas.

Administración de LVM

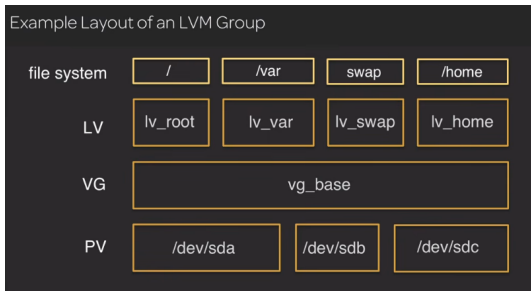


Figure: Estructura de LVM

Comandos para Physical Volumes (PV)

- ▶ `pvccreate`: Inicializa un disco o partición como PV.
- ▶ `pvs`: Muestra un resumen de los PV existentes.
- ▶ `pvddisplay`: Muestra información detallada de un PV.

102.2 Instalar un gestor de arranque

El Cargador de Arranque

Es el primer software que se ejecuta al encender la PC. Su propósito es cargar el kernel de Linux y entregarle el control.

GRUB (Grand Unified Bootloader)

- ▶ Es el gestor de arranque más popular en Linux.
- ▶ Permite elegir entre múltiples sistemas operativos y versiones del kernel.
- ▶ Se puede editar en el momento del arranque para pasar parámetros al kernel o solucionar fallos.

GRUB Legacy vs. GRUB 2

GRUB Legacy

- ▶ Versión original de GRUB (desarrollada en 1995).
- ▶ Ya no tiene desarrollo activo (última versión 0.97 en 2005).
- ▶ Archivo de configuración: `/boot/grub/menu.lst`

GRUB 2

- ▶ Reescritura completa de GRUB, más moderno y potente.
- ▶ Soporte para temas, más sistemas de archivos, UUIDs, y un archivo de configuración similar a un script.
- ▶ Archivo de configuración: `/boot/grub/grub.cfg`

¿Dónde se ubica el cargador de arranque?

BIOS (MBR)

- ▶ GRUB se instala en el primer sector del disco, el **MBR** (Master Boot Record).
- ▶ La primera etapa carga una segunda etapa desde el espacio entre el MBR y la primera partición.

UEFI (GPT)

- ▶ El firmware UEFI carga GRUB directamente desde la **EFI System Partition (ESP)**.
- ▶ El archivo es `grubx64.efi` (para sistemas de 64 bits).

La Partición /boot

¿Qué contiene?

- ▶ **Archivo de configuración** (`config-$(uname -r)`)
- ▶ **Mapa del sistema** (`System.map-$(uname -r)`)
- ▶ **Kernel de Linux** (`vmlinuz-$(uname -r)`)
- ▶ **Disco RAM inicial** (`initrd.img-$(uname -r)`)
- ▶ **Archivos de GRUB** (`/boot/grub/` o `/boot/grub2/`)

Importante

Si bien no es estrictamente necesaria, una partición /boot separada es una buena práctica por seguridad y compatibilidad.

Instalando GRUB 2

Comando grub-install

- ▶ Instala GRUB en el dispositivo de arranque.
- ▶ Ejemplo: `grub-install --boot-directory=/boot /dev/sda`
- ▶ En sistemas con partición /boot separada: `grub-install --boot-directory=/mnt/boot /dev/sda`

Actualización desde un sistema en vivo

- 1 Montar la partición raíz y /boot si es necesario.
- 2 Ejecutar `grub-install` apuntando al dispositivo correcto.
- 3 Ejecutar `update-grub` para regenerar la configuración.

Configurando GRUB 2

Archivos y Comandos Clave

- ▶ `/etc/default/grub`: Archivo principal de configuración.
- ▶ `update-grub`: Comando para generar el archivo `grub.cfg` (equivalente a `grub-mkconfig -o /boot/grub/grub.cfg`).

Configuración Común

- ▶ `GRUB_DEFAULT`: Entrada del menú que se carga por defecto.
- ▶ `GRUB_TIMEOUT`: Tiempo de espera en segundos antes de iniciar el sistema por defecto.
- ▶ `GRUB_CMDLINE_LINUX`: Parámetros adicionales para el kernel.

Interactuando con GRUB 2

En el arranque

- ▶ Presionar Shift (BIOS) o Esc (UEFI) para mostrar el menú.
- ▶ Presionar e para editar una entrada.
- ▶ Presionar c para entrar al shell de GRUB.

Arranque desde el shell de GRUB

- ❶ ls para listar particiones.
- ❷ set root=(hd0,msdos1) para definir la partición de arranque.
- ❸ linux /vmlinuz root=/dev/sda1 ro para cargar el kernel.
- ❹ initrd /initrd.img para cargar el disco RAM inicial.
- ❺ boot para iniciar el sistema.

102.3 Gestión de librerías compartidas

Librerías Compartidas (Shared Libraries)

- ▶ Son partes de código compilado y reutilizable (funciones, clases) que varios programas usan de manera común.
- ▶ También se conocen como **objetos compartidos** o **archivos .so**.

Ventaja

Al ser compartidas, ocupan menos espacio en disco y en memoria, y permiten actualizar la biblioteca sin recompilar todos los programas que la usan.

Convenciones de Nomenclatura

Formato de Nombre

Un archivo de librería compartida sigue el patrón:

libnombre.so.versión

Ejemplos

- ▶ `libc.so.6` (Librería estándar de C)
- ▶ `libpthread.so.0` (Librería de hilos POSIX)

Ubicaciones Comunes

- ▶ `/lib`
- ▶ `/lib64`
- ▶ `/usr/lib`
- ▶ `/usr/local/lib`

Configuración de Rutas y el Enlazador Dinámico

ldconfig

- ▶ Lee la configuración de rutas de librerías (normalmente en `/etc/ld.so.conf.d/`).
- ▶ Actualiza el caché `/etc/ld.so.cache` para que el enlazador dinámico encuentre las librerías rápidamente.
- ▶ Debe ejecutarse después de agregar o actualizar librerías.

Variable de Entorno LD_LIBRARY_PATH

- ▶ Permite agregar directorios de librerías de forma temporal para la sesión actual.
- ▶ Ejemplo: `export LD_LIBRARY_PATH=/usr/local/mylib:$LD_LIBRARY_PATH`

Buscando Dependencias con ldd

ldd

- ▶ Muestra las librerías compartidas de las que depende un programa ejecutable.
- ▶ También muestra la ruta completa de cada librería y la dirección de carga.

Ejemplo

```
$ ldd /usr/bin/git
linux-vdso.so.1 => (0x00007fffcbb31000)
libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

Nota

ldd no debe ejecutarse en binarios no confiables, ya que puede ejecutar código

Resumen de Comandos y Archivos

Archivos importantes

- ▶ `/etc/ld.so.conf` y `/etc/ld.so.conf.d/*.conf`: Definen rutas de librerías.
- ▶ `/etc/ld.so.cache`: Caché de librerías (actualizado por `ldconfig`).

Comandos clave

- ▶ `ldconfig`: Actualiza el caché de librerías.
- ▶ `ldd`: Muestra dependencias de un ejecutable.
- ▶ `modinfo`: Muestra información de un módulo del kernel.

Verificación de librerías no usadas

`ldd -u /usr/bin/git` muestra dependencias directas no utilizadas.

102.4 Gestión de paquetes Debian

Sistema de Paquetes Debian

- ▶ Utiliza paquetes con extensión `.deb`.
- ▶ Es el estándar en Debian, Ubuntu y sus derivados.
- ▶ Proporciona herramientas de bajo y alto nivel para la gestión de software.

Herramientas Principales

- ▶ **dpkg**: Herramienta de bajo nivel para instalar, eliminar y consultar paquetes `.deb`.
- ▶ **APT (Advanced Package Tool)**: Sistema de alto nivel que maneja dependencias y repositorios.

La Herramienta dpkg

Comandos Básicos de dpkg

- ▶ `dpkg -i paquete.deb`: Instala un paquete.
- ▶ `dpkg -r paquete`: Elimina un paquete (mantiene archivos de configuración).
- ▶ `dpkg -P paquete`: Elimina un paquete y purga sus archivos de configuración.
- ▶ `dpkg -I paquete.deb`: Muestra información del paquete.
- ▶ `dpkg -L paquete`: Lista los archivos instalados por el paquete.
- ▶ `dpkg-query -S /ruta/al/archivo`: Encuentra qué paquete posee un archivo.
- ▶ `dpkg-reconfigure paquete`: Reconfigura un paquete instalado.

Manejo de Dependencias con dpkg

Limitación

dpkg no resuelve dependencias automáticamente. Si faltan dependencias, la instalación falla y muestra un error.

Ejemplo de error

```
dpkg: dependency problems prevent configuration of openshot-qt:  
openshot-qt depends on fonts-cantarell; however:  
Package fonts-cantarell is not installed.
```

Solución

El usuario debe encontrar e instalar las dependencias faltantes manualmente, o usar herramientas de alto nivel como apt.

Advertencia

Forzar la instalación con `--force` puede dejar el sistema en un estado inconsistente.

Advanced Package Tool (APT)

Características

- ▶ Resuelve dependencias automáticamente.
- ▶ Trabaja con repositorios de software remotos.
- ▶ Mantiene una caché local de paquetes descargados (`/var/cache/apt/archives/`).

Comandos Principales de apt-get

- ▶ `apt-get update`: Actualiza la lista de paquetes desde los repositorios.
- ▶ `apt-get install paquete`: Instala un paquete y sus dependencias.
- ▶ `apt-get remove paquete`: Elimina un paquete.
- ▶ `apt-get purge paquete`: Elimina un paquete y sus configuraciones.
- ▶ `apt-get upgrade`: Actualiza todos los paquetes del sistema.

Comandos de Consulta de APT

apt-cache

- ▶ `apt-cache search patron`: Busca paquetes que coincidan con el patrón (en nombre o descripción).
- ▶ `apt-cache show paquete`: Muestra información detallada del paquete.

apt-file

- ▶ Permite buscar archivos dentro de paquetes **incluso si no están instalados**.
- ▶ `apt-file update`: Actualiza la caché de apt-file.
- ▶ `apt-file search archivo`: Encuentra qué paquete contiene un archivo.
- ▶ `apt-file list paquete`: Lista los archivos de un paquete.

El comando apt (moderno)

- ▶ Combina lo más usado de `apt-get` y `apt-cache`

La Lista de Fuentes: `/etc/apt/sources.list`

Propósito

Define los repositorios de software que APT utilizará para descargar e instalar paquetes.

Sintaxis de una Línea

```
deb http://url.repositorio.com distribucion  
componentes
```

Ejemplo (Ubuntu)

```
deb http://us.archive.ubuntu.com/ubuntu/ disco main rest
```

- ▶ **deb**: Tipo de paquetes (binarios). `deb-src` para código fuente.
- ▶ **URL**: Dirección del repositorio.
- ▶ **disco**: Nombre en clave de la distribución.

Componentes en Debian y Ubuntu

Componentes en Ubuntu

- ▶ **main:** Software libre con soporte oficial.
- ▶ **restricted:** Software de código cerrado con soporte oficial (ej. drivers).
- ▶ **universe:** Software libre mantenido por la comunidad.
- ▶ **multiverse:** Software no libre o con patentes.

Componentes en Debian

- ▶ **main:** Paquetes que cumplen con las DFSG (Debian Free Software Guidelines).
- ▶ **contrib:** Paquetes DFSG que dependen de software no DFSG.
- ▶ **non-free:** Paquetes que no son DFSG.
- ▶ **security:** Actualizaciones de seguridad.
- ▶ **backports:** Versiones más recientes de paquetes.

Directorio `/etc/apt/sources.list.d/`

Ventaja

Permite agregar archivos `.list` con repositorios adicionales, manteniendo `sources.list` más limpio y organizado.

Ejemplo

Archivo: `/etc/apt/sources.list.d/buster-backports.list`

```
deb http://deb.debian.org/debian buster-backports main c
deb-src http://deb.debian.org/debian buster-backports ma
```

Procedimiento

- ➊ Agregar el archivo `.list` en `/etc/apt/sources.list.d/`.
- ➋ Ejecutar `apt-get update` para actualizar el índice.
- ➌ Instalar paquetes con `apt-get install`.

102.5 Gestión de paquetes RPM y YUM

Sistema de Paquetes RPM

- ▶ Desarrollado por Red Hat. Es el estándar en Red Hat Enterprise Linux (RHEL), Fedora, CentOS, openSUSE, etc.
- ▶ Los paquetes tienen extensión `.rpm`.
- ▶ Un paquete hecho para una distribución puede no funcionar en otra (diferencias internas).

Herramientas Principales

- ▶ **rpm**: Herramienta de bajo nivel para instalar, actualizar y eliminar paquetes.
- ▶ **YUM (Yellowdog Updater Modified)**: Herramienta de alto nivel con resolución de dependencias.
- ▶ **DNF (Dandified YUM)**: Evolución de YUM, usado en Fedora y RHEL 8+.
- ▶ **Zypper**: Herramienta usada en SUSE Linux y openSUSE

El Gestor de Paquetes RPM (rpm)

Comandos Básicos de Instalación y Actualización

- ▶ `rpm -i paquete.rpm`: Instala un paquete.
- ▶ `rpm -U paquete.rpm`: Actualiza un paquete (o lo instala si no existe).
- ▶ `rpm -F paquete.rpm`: Solo actualiza si ya está instalado.
- ▶ `rpm -e paquete`: Elimina un paquete.
- ▶ Opciones comunes: `-v` (verbose), `-h` (hash marks).

Consultas con rpm

- ▶ `rpm -qa`: Lista todos los paquetes instalados.
- ▶ `rpm -qi paquete`: Muestra información del paquete.
- ▶ `rpm -ql paquete`: Lista los archivos de un paquete instalado.
- ▶ `rpm -qf /ruta/al/archivo`: Encuentra qué paquete posee un archivo.

Manejo de Dependencias con rpm

Limitación

rpm no resuelve dependencias automáticamente. Si al instalar faltan dependencias, la instalación falla.

Ejemplo de error

```
# rpm -i gimp-2.8.22-1.el7.x86_64.rpm  
error: Failed dependencies:  
libgimpui-2.0.so.0()(64bit) is needed by gimp-2:2.8.22-1
```

Solución

El usuario debe buscar e instalar las dependencias manualmente, o usar herramientas de alto nivel como yum o dnf.

YellowDog Updater Modified (YUM)

Características

- ▶ Herramienta de alto nivel que maneja repositorios y dependencias.
- ▶ Similar en función a apt.
- ▶ Archivos de configuración: `/etc/yum.conf` y `/etc/yum.repos.d/*.repo`.

Comandos Principales de yum

- ▶ `yum install paquete`: Instala un paquete y sus dependencias.
- ▶ `yum update paquete`: Actualiza un paquete.
- ▶ `yum update`: Actualiza todos los paquetes del sistema.
- ▶ `yum remove paquete`: Elimina un paquete.
- ▶ `yum search patron`: Busca paquetes.
- ▶ `yum info paquete`: Muestra información de un paquete.

Gestión de Repositorios con YUM

Archivos .repo

- ▶ Ubicados en `/etc/yum.repos.d/`.
- ▶ Cada repositorio tiene un archivo `.repo` con la URL, nombre y configuración.

Comandos de gestión

- ▶ `yum repolist all`: Lista todos los repositorios (habilitados y deshabilitados).
- ▶ `yum-config-manager --add-repo URL`: Agrega un repositorio.
- ▶ `yum-config-manager --enable ID`: Habilita un repositorio.
- ▶ `yum-config-manager --disable ID`: Deshabilita un repositorio.
- ▶ `yum clean packages`: Limpia la caché de paquetes descargados.

DNF (Dandified YUM)

Características

- ▶ Es la evolución moderna de YUM, usado en Fedora y RHEL 8+.
- ▶ Sintaxis y comandos muy similares a YUM.
- ▶ Mejor rendimiento y más funcionalidades.

Comandos Principales de dnf

- ▶ `dnf install paquete`: Instala un paquete.
- ▶ `dnf update paquete`: Actualiza un paquete.
- ▶ `dnf remove paquete`: Elimina un paquete.
- ▶ `dnf search patron`: Busca paquetes.
- ▶ `dnf info paquete`: Muestra información.
- ▶ `dnf provides /ruta/al/archivo`: Encuentra qué paquete proporciona un archivo.
- ▶ `dnf repolist`: Lista repositorios

Zypper (SUSE Linux)

Características

- ▶ Herramienta de gestión de paquetes para SUSE Linux y openSUSE.
- ▶ Funciona con repositorios y resuelve dependencias automáticamente.

Comandos Principales de zypper

- ▶ `zypper refresh`: Actualiza los índices de repositorios.
- ▶ `zypper install paquete`: Instala un paquete.
- ▶ `zypper update`: Actualiza todos los paquetes.
- ▶ `zypper remove paquete`: Elimina un paquete.
- ▶ `zypper search patron`: Busca paquetes.
- ▶ `zypper info paquete`: Muestra información.
- ▶ `zypper se --provides /ruta/al/archivo`: Encuentra qué paquete proporciona un archivo.

Gestión de Repositorios con Zypper

Comandos de gestión

- ▶ `zypper repos` o `zypper lr`: Lista repositorios.
- ▶ `zypper addrepo URL nombre`: Agrega un repositorio.
- ▶ `zypper removerepo nombre`: Elimina un repositorio.
- ▶ `zypper modifyrepo -d nombre`: Deshabilita un repositorio.
- ▶ `zypper modifyrepo -e nombre`: Habilita un repositorio.
- ▶ `zypper modifyrepo -f nombre`: Habilita actualización automática.
- ▶ `zypper modifyrepo -F nombre`: Deshabilita actualización automática.

Ubicación

Los repositorios se almacenan en archivos `.repo` en `/etc/zypp/repos.d/`.

102.6 Linux como sistema virtualizado

Virtualización

Permite ejecutar múltiples sistemas operativos (máquinas virtuales) de forma aislada en un solo servidor físico, optimizando el uso de recursos.

Hipervisor

Es el software encargado de gestionar y ejecutar las máquinas virtuales.

- ▶ **Tipo 1 (Bare-metal):** Se ejecuta directamente sobre el hardware (ej. Xen, KVM).
- ▶ **Tipo 2 (Hosted):** Se ejecuta sobre un sistema operativo anfitrión (ej. VirtualBox).

Tipos de Máquinas Virtuales

Totalmente Virtualizado (HardwareVM)

- ▶ El invitado no sabe que es una máquina virtual.
- ▶ Necesita extensiones de CPU (Intel VT-x o AMD-V).
- ▶ No requiere modificaciones en el sistema operativo invitado.

Paravirtualizado (PVM)

- ▶ El invitado es consciente de ser una VM.
- ▶ Usa un kernel modificado y controladores especiales (guest drivers) para mejorar el rendimiento.
- ▶ Ejemplo: Virtio para KVM.

Híbrido (HVM con PV drivers)

- ▶ Combina virtualización completa para la CPU con controladores paravirtualizados para E/S (disco y red).
- ▶ Ofrece alto rendimiento sin modificar el SO invitado.

Hipervisores Comunes en Linux

Xen

- ▶ Hipervisor de tipo 1 (bare-metal).
- ▶ Soporta paravirtualización y virtualización completa.
- ▶ Usado en entornos de nube y servidores.

KVM (Kernel Virtual Machine)

- ▶ Módulo del kernel de Linux que convierte al kernel en un hipervisor.
- ▶ Soporta virtualización completa con aceleración por hardware.
- ▶ Se gestiona con `libvirt` y herramientas como `virsh` o `virt-manager`.

VirtualBox

- ▶ Hipervisor de tipo 2 (hosted), multiplataforma.
- ▶ Popular para entornos de escritorio y desarrollo.

Ejemplo de Máquina Virtual con KVM y libvirt

Archivos de Configuración

- ▶ **XML**: Define el hardware de la VM (RAM, CPU, red, dispositivos). Ubicado en `/etc/libvirt/qemu/`.
- ▶ **Imagen de disco**: Archivo que contiene el SO y los datos.

Formatos de Imagen

- ▶ **qcow2** (Copy-on-Write): Thin-provisioning, solo ocupa el espacio usado.
- ▶ **raw**: Preasigna todo el espacio, mejor rendimiento pero ocupa más.

Ejemplo de definición XML

```
<domain type='kvm'>
  <name>rhel8.0</name>
  <memory unit='KiB'>4194304</memory>
  <vcpu>2</vcpu>
```

Plantillas y Clonación de Máquinas Virtuales

Uso de Plantillas

- ▶ Se crea una VM base con la instalación y configuración estándar.
- ▶ Se clona para desplegar nuevas VMs rápidamente.
- ▶ Reduce el tiempo de instalación y configuración repetitiva.

Precauciones al Clonar

- ▶ La **ID de máquina D-Bus** debe ser única para cada sistema.
- ▶ Se almacena en `/etc/machine-id` y `/var/lib/dbus/machine-id`.
- ▶ Se deben regenerar las **claves de host SSH** para evitar conflictos de seguridad.

Generar nueva ID D-Bus

```
# rm -f /etc/machine-id  
# dbus-uuidgen --ensure=/etc/machine-id
```

Máquinas Virtuales en la Nube (IaaS)

Elementos Clave

- ▶ **Instancias de Computación:** VMs que se cobran por tiempo de uso.
- ▶ **Almacenamiento en Bloque:** Discos virtuales con diferentes niveles de rendimiento y costo.
- ▶ **Redes Virtuales:** Configuración de subredes, rutas, firewalls y DNS.

Acceso Seguro

- ▶ Uso de **SSH** con claves públicas/privadas.
- ▶ `ssh-keygen` para generar el par de claves.
- ▶ `ssh-copy-id` para copiar la clave pública al servidor remoto.

Preconfiguración con cloud-init

cloud-init

- ▶ Herramienta para preconfigurar VMs en la nube usando archivos YAML.
- ▶ Se ejecuta durante el primer arranque de la VM.
- ▶ Independiente del proveedor de nube.

Ejemplo de archivo cloud-config

```
#cloud-config
timezone: Africa/Dar_es_Salaam
hostname: test-system
apt_update: true
apt_upgrade: true
packages:
- nginx
```

Contenedores Linux

Contenedores vs. Máquinas Virtuales

- ▶ Un contenedor es un entorno aislado a nivel de sistema operativo que comparte el kernel del host.
- ▶ Son más ligeros que las VMs (menos overhead) y arrancan mucho más rápido.
- ▶ Utilizan mecanismos del kernel como **cgroups** y **namespaces** para el aislamiento de recursos.

Tecnologías de Contenedores

- ▶ **Docker**: El estándar de facto para contenedores de aplicaciones.
- ▶ **LXC/LXD**: Contenedores a nivel de sistema, similares a VMs ligeras.
- ▶ **systemd-nspawn**: Contenedores nativos de systemd.
- ▶ **Kubernetes**: Orquestación de contenedores a gran escala

¿Preguntas o comentarios?